

ZONE 3: RUN - Live Operations & Response

Technical Specifications for eVenu360 MVP

Overview

Temporal Frame: T-0 (Gates Open) → T+End (Event Conclusion)
Primary Function: Real-time operational command, incident management, and automated response orchestration
Key Outputs: Live dashboards, incident reports, response coordination, evidence capture

1. DATABASE SCHEMA

Core Tables

typescript

```
TypeScript
// =====
// LIVE OPERATIONS TABLES
// =====

interface LiveEvent {
  event_id: string;           // PK: uuid
  venue_id: string;           // FK: venues.venue_id
  event_name: string;
  event_type: 'nfl_game' | 'concert' | 'conference' | 'other';
  event_status: 'pre_gates' | 'gates_open' | 'active' |
'halftime' | 'post_event' | 'closed';
  gates_opened_at: timestamp;
  event_started_at: timestamp;
  event_ended_at: timestamp;
```

```

    expected_attendance: number;
    actual_attendance: number;
    current_occupancy: number;
    weather_conditions: {
      temperature: number;
      conditions: string;
      wind_speed: number;
      precipitation: number;
      alerts: string[];
    };
    operational_phase: 'ingress' | 'pre_event' | 'live_event' |
'egress';
  }

interface LiveMetrics {
  metric_id: string;           // PK: uuid
  event_id: string;           // FK: live_events.event_id
  timestamp: timestamp;
  metric_type: 'attendance' | 'flow' | 'wait_time' |
'transaction' | 'system_health';
  metric_data: {
    // Attendance metrics
    total_entered?: number;
    entry_rate_per_minute?: number;
    zones_occupied?: {
      zone_id: string;
      occupancy: number;
      capacity_percentage: number;
    }[];

    // Flow metrics
    crowd_density?: {
      zone_id: string;
      density: number;           // people per sq meter
      flow_rate: number;        // people per minute
      congestion_level: 'low' | 'medium' | 'high' | 'critical';
    };
  };
};

```

```

    }[];

    // Wait time metrics
    concession_wait_times?: {
        location_id: string;
        avg_wait_minutes: number;
        queue_length: number;
    }[];
    entry_wait_times?: {
        gate_id: string;
        avg_wait_minutes: number;
        queue_length: number;
    }[];

    // Transaction metrics
    transactions_per_minute?: number;
    avg_transaction_value?: number;
    revenue_rate?: number;

    // System health
    systems_operational?: number;
    systems_degraded?: number;
    systems_offline?: number;
};
}

interface Incident {
    incident_id: string; // PK: uuid
    event_id: string; // FK: live_events.event_id
    venue_id: string; // FK: venues.venue_id
    incident_number: string; // e.g., "INC-2025-001"
    incident_type: 'medical' | 'security' | 'safety' |
'operational' | 'guest_service' | 'technical';
    severity: 'critical' | 'high' | 'medium' | 'low';
    status: 'reported' | 'acknowledged' | 'responding' | 'resolved'
| 'closed';

```

```

    reported_at: timestamp;
    acknowledged_at: timestamp;
    resolved_at: timestamp;
    closed_at: timestamp;
    location: {
      zone_id: string;
      section: string;
      row?: string;
      seat?: string;
      latitude: number;
      longitude: number;
      floor_level: number;
    };
    description: string;
    reporter: {
      reporter_type: 'staff' | 'guest' | 'automated' | 'external';
      reporter_id: string;
      contact_info?: string;
    };
    assigned_to: string[];           // FK: staff.staff_id[]
    response_team: string;           // Team assigned
    estimated_resolution_time: timestamp;
    actual_resolution_time: timestamp;
    escalation_level: number;        // 0-3
    escalated_to: string[];          // FK: staff.staff_id[]
    tags: string[];
  }

interface IncidentTimeline {
  timeline_id: string;              // PK: uuid
  incident_id: string;              // FK: incidents.incident_id
  timestamp: timestamp;
  event_type: 'reported' | 'acknowledged' | 'dispatched' |
'arrived' | 'action_taken' | 'resolved' | 'note_added';
  actor: {
    actor_type: 'staff' | 'system' | 'external';

```

```

    actor_id: string;
    actor_name: string;
  };
  details: string;
  location_at_time?: {
    latitude: number;
    longitude: number;
  };
  evidence_captured?: {
    photos: string[];
    videos: string[];
    audio: string[];
    sensor_data: jsonb;
  };
}

interface ResponseProtocol {
  protocol_id: string;           // PK: uuid
  incident_id: string;          // FK: incidents.incident_id
  protocol_type: 'evacuation' | 'medical_emergency' |
'security_threat' | 'fire' | 'weather';
  protocol_name: string;
  activated_at: timestamp;
  activated_by: string;          // FK: staff.staff_id
  protocol_steps: {
    step_number: number;
    step_name: string;
    assigned_to: string;          // Role or specific staff
    status: 'pending' | 'in_progress' | 'completed' | 'skipped';
    started_at: timestamp;
    completed_at: timestamp;
    notes: string;
  }[];
  agencies_notified: {
    agency: string;
    notification_method: string;
  };
}

```

```

    notified_at: timestamp;
    acknowledged: boolean;
  }[];
  resources_deployed: {
    resource_type: string;
    resource_id: string;
    deployed_at: timestamp;
    location: string;
  }[];
  completion_status: 'active' | 'completed' | 'aborted';
  deactivated_at: timestamp;
}

interface CrowdFlowAnalysis {
  analysis_id: string;           // PK: uuid
  event_id: string;              // FK: live_events.event_id
  timestamp: timestamp;
  analysis_type: 'real_time' | 'predictive' | 'historical';
  zones_analyzed: {
    zone_id: string;
    current_density: number;      // people per sq meter
    density_trend: 'increasing' | 'stable' | 'decreasing';
    predicted_density_5min: number;
    predicted_density_15min: number;
    congestion_risk: 'low' | 'medium' | 'high' | 'critical';
    recommended_actions: string[];
  }[];
  flow_patterns: {
    from_zone: string;
    to_zone: string;
    flow_rate: number;           // people per minute
    bottleneck_detected: boolean;
  }[];
  alerts_generated: {
    alert_type: 'density_threshold' | 'bottleneck' |
    'unsafe_flow';
  }
}
```

```

        zone_id: string;
        severity: 'warning' | 'critical';
        recommended_action: string;
    }[];
}

interface RealTimeAlert {
    alert_id: string;                // PK: uuid
    event_id: string;                // FK: live_events.event_id
    alert_type: 'system' | 'operational' | 'safety' | 'security' |
'predictive';
    severity: 'info' | 'warning' | 'critical' | 'emergency';
    alert_source: 'automated' | 'manual' | 'ai_prediction';
    title: string;
    message: string;
    triggered_at: timestamp;
    acknowledged_at: timestamp;
    acknowledged_by: string;        // FK: staff.staff_id
    resolved_at: timestamp;
    affected_systems: string[];
    affected_zones: string[];
    recipients: {
        recipient_type: 'role' | 'individual' | 'team';
        recipient_id: string;
        notification_method: 'push' | 'sms' | 'email' | 'radio';
        delivered_at: timestamp;
        acknowledged: boolean;
    }[];
    auto_response_triggered: boolean;
    auto_response_actions: string[];
    priority: number;                // 1-10
    expires_at: timestamp;
}

interface StaffLocation {
    location_id: string;            // PK: uuid

```

```

    staff_id: string;           // FK: staff.staff_id
    event_id: string;          // FK: live_events.event_id
    timestamp: timestamp;
    location: {
        latitude: number;
        longitude: number;
        floor_level: number;
        zone_id: string;
    };
    status: 'on_duty' | 'on_break' | 'responding' | 'off_duty';
    current_assignment: string;
    available: boolean;
    last_check_in: timestamp;
}

interface GuestInteraction {
    interaction_id: string;      // PK: uuid
    event_id: string;           // FK: live_events.event_id
    guest_id: string;           // Anonymized guest
    identifier
    interaction_type: 'concession_order' | 'parking' | 'wayfinding'
    | 'service_request' | 'complaint';
    timestamp: timestamp;
    channel: 'mobile_app' | 'kiosk' | 'staff' | 'website';
    location: string;
    details: jsonb;
    resolution_status: 'pending' | 'in_progress' | 'resolved';
    satisfaction_score: number;  // 1-5
    response_time_seconds: number;
}

interface EvidenceLedgerEntry {
    entry_id: string;           // PK: uuid
    event_id: string;           // FK: live_events.event_id
    incident_id: string;        // FK: incidents.incident_id
    (optional)

```



```

    timestamp: timestamp;
    evidence_type: 'photo' | 'video' | 'audio' | 'sensor_data' |
'system_log' | 'document';
    evidence_hash: string;           // SHA-256 hash for integrity
    blockchain_hash: string;        // Blockchain anchor
    storage_location: string;       // URL or path
    captured_by: {
        capture_method: 'automated' | 'manual';
        captured_by_id: string;
        device_id: string;
    };
    metadata: {
        location: jsonb;
        camera_id?: string;
        sensor_id?: string;
        file_size?: number;
        duration?: number;
        resolution?: string;
    };
    chain_of_custody: {
        timestamp: timestamp;
        action: 'captured' | 'accessed' | 'shared' | 'exported';
        actor_id: string;
        reason: string;
    }[];
    legal_hold: boolean;
    retention_until: timestamp;
}

```

```

interface SystemPerformance {
    performance_id: string;           // PK: uuid
    event_id: string;                 // FK: live_events.event_id
    system_id: string;                 // FK:
    infrastructure_systems.system_id
    timestamp: timestamp;
    metrics: {

```

```

    cpu_usage: number;
    memory_usage: number;
    disk_io: number;
    network_latency: number;
    response_time_ms: number;
    requests_per_second: number;
    error_rate: number;
    active_connections: number;
  };
  health_status: 'healthy' | 'degraded' | 'critical' | 'offline';
  alerts_triggered: string[];
  auto_scaling_events: {
    timestamp: timestamp;
    action: 'scale_up' | 'scale_down';
    reason: string;
  }[];
}

```

2. API ENDPOINTS

Live Operations API

typescript

```

TypeScript
// =====
// LIVE EVENT MANAGEMENT
// =====

/**
 * GET /api/v1/venues/:venueId/events/:eventId/live-status
 *
 * Real-time event status and metrics
 */
interface GetLiveStatusRequest {

```

```
venueId: string;
eventId: string;
}

interface GetLiveStatusResponse {
  event: LiveEvent;
  current_metrics: {
    attendance: {
      total_entered: number;
      current_occupancy: number;
      capacity_percentage: number;
      entry_rate_per_minute: number;
    };
    incidents: {
      active: number;
      critical: number;
      resolved_today: number;
      avg_resolution_time_minutes: number;
    };
    systems: {
      operational: number;
      degraded: number;
      offline: number;
      overall_health: number;
    };
    guest_experience: {
      avg_wait_time_entry: number;
      avg_wait_time_concessions: number;
      service_requests_active: number;
      satisfaction_score: number;
    };
  };
  alerts_active: RealTimeAlert[];
  weather_current: WeatherConditions;
  operational_phase: string;
}
```

```

/**
 * WebSocket: /ws/venues/:venueId/events/:eventId/live-feed
 *
 * Real-time operational data stream
 */
interface LiveFeedMessage {
  type: 'metric_update' | 'incident' | 'alert' | 'system_status'
  | 'staff_update';
  timestamp: timestamp;
  data: MetricUpdate | Incident | RealTimeAlert | SystemStatus |
  StaffUpdate;
}

// Example metric update
interface MetricUpdate {
  metric_type: 'attendance' | 'flow' | 'wait_time';
  zone_id?: string;
  value: number;
  trend: 'increasing' | 'stable' | 'decreasing';
  alert_level?: 'normal' | 'warning' | 'critical';
}

/**
 * PUT /api/v1/events/:eventId/phase
 *
 * Update operational phase
 */
interface UpdatePhaseRequest {
  eventId: string;
  new_phase: 'ingress' | 'pre_event' | 'live_event' | 'halftime'
  | 'egress' | 'closed';
  triggered_by: string;
  notes?: string;
}

```

```

interface UpdatePhaseResponse {
    event: LiveEvent;
    phase_change: {
        from: string;
        to: string;
        timestamp: timestamp;
    };
    automated_actions_triggered: string[];
    staff_notifications_sent: number;
}

// =====
// INCIDENT MANAGEMENT
// =====

/**
 * POST /api/v1/events/:eventId/incidents
 *
 * Report new incident
 */
interface ReportIncidentRequest {
    eventId: string;
    incident_type: IncidentType;
    severity: Severity;
    location: LocationData;
    description: string;
    reporter: ReporterInfo;
    media?: File[]; // Photos/videos
    immediate_response_needed?: boolean;
}

interface ReportIncidentResponse {
    incident: Incident;
    incident_number: string;
    response_team_dispatched: {
        team_name: string;
    };
}

```

```

        staff_assigned: string[];
        estimated_arrival: timestamp;
    };
    protocol_activated?: {
        protocol_name: string;
        protocol_id: string;
    };
    notifications_sent: {
        recipients: string[];
        methods: string[];
    };
}

/**
 * PUT /api/v1/incidents/:incidentId/acknowledge
 *
 * Acknowledge incident receipt
 */
interface AcknowledgeIncidentRequest {
    incidentId: string;
    acknowledged_by: string;
    estimated_response_time?: number; // minutes
    notes?: string;
}

interface AcknowledgeIncidentResponse {
    incident: Incident;
    acknowledgment_logged: boolean;
    response_timer_started: boolean;
}

/**
 * POST /api/v1/incidents/:incidentId/timeline
 *
 * Add timeline entry to incident
 */

```

```

interface AddTimelineEntryRequest {
  incidentId: string;
  event_type: TimelineEventType;
  details: string;
  actor_id: string;
  evidence?: {
    photos?: File[];
    videos?: File[];
    notes?: string;
  };
  location?: LocationData;
}

interface AddTimelineEntryResponse {
  timeline_entry: IncidentTimeline;
  evidence_entries: EvidenceLedgerEntry[];
  incident_updated: Incident;
}

/**
 * PUT /api/v1/incidents/:incidentId/resolve
 *
 * Mark incident as resolved
 */
interface ResolveIncidentRequest {
  incidentId: string;
  resolved_by: string;
  resolution_summary: string;
  resolution_actions_taken: string[];
  follow_up_required: boolean;
  follow_up_notes?: string;
  final_evidence?: File[];
}

interface ResolveIncidentResponse {
  incident: Incident;
}

```

```

    resolution_time: {
        reported_at: timestamp;
        resolved_at: timestamp;
        duration_minutes: number;
    };
    evidence_ledger_entries: EvidenceLedgerEntry[];
    auto_report_generated: boolean;
    report_id?: string;
}

/**
 * POST /api/v1/incidents/:incidentId/escalate
 *
 * Escalate incident to higher level
 */
interface EscalateIncidentRequest {
    incidentId: string;
    escalated_by: string;
    escalation_reason: string;
    escalate_to: string[];           // Staff IDs or roles
    new_severity?: Severity;
}

interface EscalateIncidentResponse {
    incident: Incident;
    escalation_logged: boolean;
    notifications_sent: {
        escalated_to: string[];
        notification_methods: string[];
    };
    protocol_changes?: {
        old_protocol?: string;
        new_protocol: string;
    };
}

```



```

/**
 * GET /api/v1/events/:eventId/incidents/active
 *
 * Get all active incidents
 */
interface GetActiveIncidentsRequest {
  eventId: string;
  filters?: {
    severity?: Severity[];
    type?: IncidentType[];
    zone?: string[];
  };
  sort_by?: 'severity' | 'time' | 'location';
}

interface GetActiveIncidentsResponse {
  incidents: Incident[];
  summary: {
    total_active: number;
    by_severity: Record<Severity, number>;
    by_type: Record<IncidentType, number>;
    avg_response_time: number;
  };
  resource_allocation: {
    staff_responding: number;
    teams_deployed: number;
    protocols_active: number;
  };
}

// =====
// RESPONSE PROTOCOLS
// =====

/**
 * POST /api/v1/incidents/:incidentId/protocols/activate

```

```

*
* Activate emergency response protocol
*/
interface ActivateProtocolRequest {
    incidentId: string;
    protocol_type: ProtocolType;
    activated_by: string;
    override_parameters?: jsonb;           // Custom protocol
modifications
    agencies_to_notify?: string[];
}

interface ActivateProtocolResponse {
    protocol: ResponseProtocol;
    protocol_steps: ProtocolStep[];
    agencies_notified: AgencyNotification[];
    resources_deploying: {
        resource_type: string;
        quantity: number;
        eta_minutes: number;
    }[];
    estimated_completion: timestamp;
}

/**
* PUT /api/v1/protocols/:protocolId/steps/:stepNumber/complete
*
* Mark protocol step as complete
*/
interface CompleteProtocolStepRequest {
    protocolId: string;
    stepNumber: number;
    completed_by: string;
    notes: string;
    evidence?: File[];
    next_step_override?: number;           // Skip to specific step

```

```

}

interface CompleteProtocolStepResponse {
  protocol: ResponseProtocol;
  step_completed: ProtocolStep;
  next_step?: ProtocolStep;
  protocol_completion_percentage: number;
  estimated_remaining_time: number;
}

/**
 * POST /api/v1/protocols/:protocolId/deactivate
 *
 * Deactivate protocol (all-clear)
 */
interface DeactivateProtocolRequest {
  protocolId: string;
  deactivated_by: string;
  deactivation_reason: 'resolved' | 'aborted' | 'superseded';
  deactivation_notes: string;
  all_clear_confirmed: boolean;
}

interface DeactivateProtocolResponse {
  protocol: ResponseProtocol;
  deactivation_logged: boolean;
  incident_updated: Incident;
  agencies_notified: string[];
  after_action_report_generated: boolean;
  report_id: string;
}

// =====
// CROWD MANAGEMENT
// =====

```

```

/**
 * GET /api/v1/events/:eventId/crowd-flow/real-time
 *
 * Real-time crowd flow analysis
 */
interface GetCrowdFlowRequest {
  eventId: string;
  zones?: string[];           // Specific zones or all
}

interface GetCrowdFlowResponse {
  timestamp: timestamp;
  analysis: CrowdFlowAnalysis;
  heat_map_data: {
    zone_id: string;
    density: number;
    color_code: string;       // For visualization
    coordinates: {
      latitude: number;
      longitude: number;
    }[];
  }[];
  alerts_active: {
    zone_id: string;
    alert_type: string;
    severity: string;
    recommended_action: string;
  }[];
  predictions: {
    time_horizon: '5min' | '15min' | '30min';
    predicted_densities: {
      zone_id: string;
      predicted_density: number;
      confidence: number;
    }[];
  };
};

```

```

}

/**
 * POST /api/v1/events/:eventId/crowd-flow/intervention
 *
 * Trigger crowd flow intervention
 */
interface TriggerInterventionRequest {
  eventId: string;
  intervention_type: 'open_gates' | 'staff_deployment' |
'route_adjustment' | 'capacity_limit';
  target_zones: string[];
  reason: string;
  triggered_by: string;
  parameters?: jsonb;
}

interface TriggerInterventionResponse {
  intervention_id: string;
  actions_taken: string[];
  staff_notified: string[];
  systems_adjusted: string[];
  expected_impact: {
    density_reduction: number;
    time_to_effect: number;
  };
  monitoring_activated: boolean;
}

/**
 * WebSocket: /ws/events/:eventId/crowd-alerts
 *
 * Real-time crowd density alerts
 */
interface CrowdAlertStream {

```

```
    type: 'density_alert' | 'bottleneck' | 'flow_disruption' |
'prediction';
    timestamp: timestamp;
    zone_id: string;
    severity: 'warning' | 'critical';
    current_density: number;
    threshold_density: number;
    predicted_density_5min: number;
    recommended_actions: string[];
    auto_intervention_triggered: boolean;
}
```

```
// =====
// STAFF COORDINATION
// =====
```

```
/**
 * GET /api/v1/events/:eventId/staff/locations
 *
 * Real-time staff location tracking
 */
```

```
interface GetStaffLocationsRequest {
    eventId: string;
    roles?: string[];
    zones?: string[];
    status?: StaffStatus[];
}
```

```
interface GetStaffLocationsResponse {
    timestamp: timestamp;
    staff_locations: StaffLocation[];
    coverage_map: {
        zone_id: string;
        staff_count: number;
        required_staff: number;
        coverage_percentage: number;
    };
}
```

```

    }[];
    available_staff: {
      role: string;
      count: number;
      locations: string[];
    }[];
  }

/**
 * POST /api/v1/events/:eventId/staff/dispatch
 *
 * Dispatch staff to incident or location
 */
interface DispatchStaffRequest {
  eventId: string;
  dispatch_reason: 'incident' | 'coverage_gap' |
'special_request';
  incident_id?: string;
  target_location: LocationData;
  required_roles: string[];
  required_count: number;
  priority: 'routine' | 'urgent' | 'emergency';
  dispatcher_id: string;
}

interface DispatchStaffResponse {
  dispatch_id: string;
  staff_assigned: {
    staff_id: string;
    staff_name: string;
    current_location: LocationData;
    eta_minutes: number;
    route: {
      latitude: number;
      longitude: number;
    }[];
  };
}

```

```

    }[];
    notifications_sent: boolean;
    tracking_enabled: boolean;
}

/**
 * PUT /api/v1/staff/:staffId/status
 *
 * Update staff status/availability
 */
interface UpdateStaffStatusRequest {
    staffId: string;
    new_status: 'on_duty' | 'on_break' | 'responding' | 'off_duty';
    location?: LocationData;
    notes?: string;
}

interface UpdateStaffStatusResponse {
    staff_location: StaffLocation;
    status_updated: boolean;
    availability_updated: boolean;
    reassignment_triggered?: {
        from_assignment: string;
        to_assignment: string;
    };
}

// =====
// ALERTS & NOTIFICATIONS
// =====

/**
 * POST /api/v1/events/:eventId/alerts
 *
 * Create manual alert
 */

```



```
interface CreateAlertRequest {
    eventId: string;
    alert_type: AlertType;
    severity: Severity;
    title: string;
    message: string;
    affected_zones?: string[];
    recipients: {
        recipient_type: 'role' | 'individual' | 'team' | 'all';
        recipient_ids?: string[];
    };
    notification_methods: ('push' | 'sms' | 'email' | 'radio')[];
    expires_in_minutes?: number;
    created_by: string;
}
```

```
interface CreateAlertResponse {
    alert: RealTimeAlert;
    notifications_queued: number;
    estimated_delivery_time: timestamp;
}
```

```
/**
 * PUT /api/v1/alerts/:alertId/acknowledge
 *
 * Acknowledge alert receipt
 */
```

```
interface AcknowledgeAlertRequest {
    alertId: string;
    acknowledged_by: string;
    action_taken?: string;
}
```

```
interface AcknowledgeAlertResponse {
    alert: RealTimeAlert;
    acknowledgment_logged: boolean;
}
```

```

    alert_auto_resolved: boolean;
}

/**
 * GET /api/v1/events/:eventId/alerts/active
 *
 * Get all active alerts
 */
interface GetActiveAlertsResponse {
    alerts: RealTimeAlert[];
    summary: {
        total_active: number;
        by_severity: Record<Severity, number>;
        unacknowledged: number;
    };
}

// =====
// EVIDENCE LEDGER
// =====

/**
 * POST /api/v1/events/:eventId/evidence
 *
 * Capture evidence entry
 */
interface CaptureEvidenceRequest {
    eventId: string;
    incident_id?: string;
    evidence_type: EvidenceType;
    files: File[];
    metadata: {
        location: LocationData;
        camera_id?: string;
        sensor_id?: string;
        description: string;
    };
}

```

```

    };
    captured_by: string;
    legal_hold?: boolean;
}

interface CaptureEvidenceResponse {
    evidence_entries: EvidenceLedgerEntry[];
    blockchain_anchors: {
        entry_id: string;
        hash: string;
        blockchain_transaction: string;
        timestamp: timestamp;
    }[];
    storage_locations: {
        entry_id: string;
        url: string;
    }[];
}

/**
 * GET /api/v1/evidence/:entryId/chain-of-custody
 *
 * Get complete chain of custody
 */
interface GetChainOfCustodyResponse {
    evidence_entry: EvidenceLedgerEntry;
    chain_of_custody: CustodyRecord[];
    integrity_verified: boolean;
    blockchain_verified: boolean;
    legal_admissibility: 'admissible' | 'questionable' |
'compromised';
}

// =====
// SYSTEM MONITORING
// =====

```

```

/**
 * GET /api/v1/events/:eventId/systems/performance
 *
 * Real-time system performance metrics
 */
interface GetSystemPerformanceRequest {
  eventId: string;
  systems?: string[];
  time_range?: {
    start: timestamp;
    end: timestamp;
  };
}

interface GetSystemPerformanceResponse {
  timestamp: timestamp;
  systems: SystemPerformance[];
  overall_health: {
    score: number;
    status: 'healthy' | 'degraded' | 'critical';
    systems_healthy: number;
    systems_degraded: number;
    systems_critical: number;
  };
  performance_trends: {
    system_id: string;
    trend: 'improving' | 'stable' | 'degrading';
    forecast_1hour: number;
  }[];
}

/**
 * WebSocket: /ws/events/:eventId/system-health
 *
 * Real-time system health monitoring

```

```

    */
interface SystemHealthStream {
    type: 'performance_update' | 'health_change' | 'alert' |
    'auto_scaling';
    timestamp: timestamp;
    system_id: string;
    data: SystemPerformance | HealthChange | SystemAlert |
    ScalingEvent;
}

```

3. UI COMPONENTS

Live Command Center

typescript

```

TypeScript
// =====
// MAIN COMMAND CENTER COMPONENT
// =====

interface CommandCenterProps {
    venueId: string;
    eventId: string;
    userRole: UserRole;
}

const LiveCommandCenter: React.FC<CommandCenterProps> = ({
    venueId,
    eventId,
    userRole
}) => {
    // WebSocket connections

```

```

    const { liveFeed } =
useWebSocket(`/ws/venues/${venueId}/events/${eventId}/live-feed`)
;
    const { crowdAlerts } =
useWebSocket(`/ws/events/${eventId}/crowd-alerts`);
    const { systemHealth } =
useWebSocket(`/ws/events/${eventId}/system-health`);

return (
  <CommandCenterLayout>
    {/* Top Status Bar */}
    <StatusBar
      eventStatus={eventStatus}
      currentPhase={operationalPhase}
      attendance={attendanceMetrics}
      alerts={activeAlerts.length}
    />

    {/* Main Grid Layout */}
    <Grid container spacing={2}>
      {/* Left Column: Incidents & Alerts */}
      <Grid item xs={12} lg={4}>
        <Stack spacing={2}>
          <ActiveIncidentsPanel
            incidents={activeIncidents}
            onSelectIncident={handleSelectIncident}
            onReportIncident={handleReportIncident}
          />

          <ActiveAlertsPanel
            alerts={activeAlerts}
            onAcknowledge={handleAcknowledgeAlert}
          />
        </Stack>
      </Grid>

```

```

    {/* Center: Live Venue Map */}
    <Grid item xs={12} lg={4}>
      <LiveVenueMap
        venue={venueData}
        crowdDensity={crowdFlowData}
        incidents={activeIncidents}
        staffLocations={staffLocations}
        onZoneClick={handleZoneClick}
      />
    </Grid>

    {/* Right Column: Metrics & Systems */}
    <Grid item xs={12} lg={4}>
      <Stack spacing={2}>
        <LiveMetricsPanel
          attendance={attendanceMetrics}
          waitTimes={waitTimeMetrics}
          transactions={transactionMetrics}
        />

        <SystemHealthPanel
          systems={systemsHealth}
          alerts={systemAlerts}
        />
      </Stack>
    </Grid>
  </Grid>

  {/* Bottom: Recent Activity Timeline */}
  <ActivityTimeline
    events={recentActivity}
    autoScroll={true}
  />

  {/* Modals */}
  {selectedIncident && (

```

```

        <IncidentDetailModal
            incident={selectedIncident}
            onClose={() => setSelectedIncident(null)}
            onUpdate={handleUpdateIncident}
        />
    )}
</CommandCenterLayout>
);
};

// =====
// ACTIVE INCIDENTS PANEL
// =====

interface ActiveIncidentsPanelProps {
    incidents: Incident[];
    onSelectIncident: (incident: Incident) => void;
    onReportIncident: () => void;
}

const ActiveIncidentsPanel: React.FC<ActiveIncidentsPanelProps> =
({
    incidents,
    onSelectIncident,
    onReportIncident
}) => {
    const criticalIncidents = incidents.filter(i => i.severity ===
'critical');
    const highIncidents = incidents.filter(i => i.severity ===
'high');

    return (
        <Card sx={{ height: '100%' }}>
            <CardHeader
                title={
                    <Box display="flex" alignItems="center" gap={1}>

```



```

        <Typography variant="h6">Active
Incidents</Typography>
        <Chip
          label={incidents.length}
          size="small"
          color={criticalIncidents.length > 0 ? 'error' :
'default'}
        />
      </Box>
    }
    action={
      <Button
        variant="contained"
        color="error"
        startIcon={<AddIcon />}
        onClick={onReportIncident}
      >
        Report
      </Button>
    }
    sx={{ background: '#1B3A4B', color: 'white' }}
  />
<CardContent sx={{ maxHeight: '500px', overflow: 'auto' }}>
  <List>
    {/* Critical Incidents */}
    {criticalIncidents.length > 0 && (
      <>
        <ListSubheader sx={{ background: '#D0021B', color:
'white' }}>
          CRITICAL ({criticalIncidents.length})
        </ListSubheader>
        {criticalIncidents.map(incident => (
          <IncidentListItem
            key={incident.incident_id}
            incident={incident}
            onClick={() => onSelectIncident(incident)}

```

```

        />
      )}}
    </>
  )}

  {/* High Priority Incidents */}
  {highIncidents.length > 0 && (
    <>
      <ListSubheader sx={{ background: '#F5A623', color:
'white' }}>
        HIGH PRIORITY ({highIncidents.length})
      </ListSubheader>
      {highIncidents.map(incident => (
        <IncidentListItem
          key={incident.incident_id}
          incident={incident}
          onClick={() => onSelectIncident(incident)}
        />
      )}}
    </>
  )}

  {/* Other Incidents */}
  {incidents.filter(i => ![ 'critical',
'high' ].includes(i.severity)).map(incident => (
    <IncidentListItem
      key={incident.incident_id}
      incident={incident}
      onClick={() => onSelectIncident(incident)}
    />
  )}}
  </List>
</CardContent>
</Card>
);
};

```

```

// =====
// INCIDENT LIST ITEM
// =====

interface IncidentListItemProps {
  incident: Incident;
  onClick: () => void;
}

const IncidentListItem: React.FC<IncidentListItemProps> = ({
  incident,
  onClick
}) => {
  const getStatusColor = (status: string) => {
    switch (status) {
      case 'reported': return '#F5A623';
      case 'acknowledged': return '#3A7B8A';
      case 'responding': return '#4A90E2';
      case 'resolved': return '#9AAF45';
      default: return '#5A6A72';
    }
  };

  const getTimeElapsed = (timestamp: Date) => {
    const now = new Date();
    const diff = now.getTime() - new Date(timestamp).getTime();
    const minutes = Math.floor(diff / 60000);
    return minutes < 60 ? `${minutes}m` : `${Math.floor(minutes / 60)}h ${minutes % 60}m`;
  };

  return (
    <ListItem
      button
      onClick={onClick}

```

```

sx={{
  border: '1px solid #D1D9DD',
  borderRadius: 1,
  mb: 1,
  borderLeft: `4px solid
${getSeverityColor(incident.severity)}`,
  '&:hover': { backgroundColor: '#F4F6F7' }
}}
>
<ListItemText
  primary={
    <Box display="flex" justifyContent="space-between"
alignItems="center">
      <Typography variant="subtitle2" fontWeight="bold">
        {incident.incident_number}
      </Typography>
      <Chip
        label={incident.status.toUpperCase()}
        size="small"
        sx={{
          backgroundColor: getStatusColor(incident.status),
          color: 'white'
        }}
      />
    </Box>
  }
  secondary={
    <Box mt={0.5}>
      <Typography variant="body2" noWrap>
        {incident.description}
      </Typography>
      <Box display="flex" gap={1} mt={0.5}>
        <Chip
          icon={<LocationIcon />}
          label={` ${incident.location.zone_id} -
${incident.location.section}`}

```

```

        size="small"
        variant="outlined"
    />
    <Chip
        icon={<TimerIcon />}
        label={getTimeElapsed(incident.reported_at)}
        size="small"
        variant="outlined"
    />
    {incident.assigned_to.length > 0 && (
        <Chip
            icon={<PersonIcon />}
            label={`${incident.assigned_to.length}
assigned`}
            size="small"
            variant="outlined"
        />
    )}
    </Box>
</Box>
}
/>
</ListItem>
);
};

```

```

// =====
// LIVE VENUE MAP
// =====

```

```

interface LiveVenueMapProps {
    venue: VenueBlueprint;
    crowdDensity: CrowdFlowAnalysis;
    incidents: Incident[];
    staffLocations: StaffLocation[];
    onZoneClick: (zoneId: string) => void;
}

```

```

}

const LiveVenueMap: React.FC<LiveVenueMapProps> = ({
  venue,
  crowdDensity,
  incidents,
  staffLocations,
  onZoneClick
}) => {
  return (
    <Card sx={{ height: '600px' }}>
      <CardHeader
        title="Live Venue Status"
        action={
          <ToggleButtonGroup size="small" exclusive
value={mapLayer}>
            <ToggleButton value="crowd">Crowd</ToggleButton>
            <ToggleButton
value="incidents">Incidents</ToggleButton>
            <ToggleButton value="staff">Staff</ToggleButton>
          </ToggleButtonGroup>
        }
        sx={{ background: '#3A7B8A', color: 'white' }}
      />
      <CardContent sx={{ height: 'calc(100% - 64px)', p: 0 }}>
        <MapContainer
          center={venue.center_coordinates}
          zoom={18}
          style={{ height: '100%', width: '100%' }}
        >
          <TileLayer
url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" />

          {/* Venue Zones with Crowd Density */}
          {crowdDensity.zones_analyzed.map(zone => (
            <Polygon

```

```

        key={zone.zone_id}
        positions={getZonePolygon(zone.zone_id)}
        pathOptions={{
          fillColor: getDensityColor(zone.current_density),
          fillOpacity: 0.6,
          color: zone.congestion_risk === 'critical' ?
'#D0021B' : '#1B3A4B',
          weight: zone.congestion_risk === 'critical' ? 3 :
1
        }}
        eventHandlers={{
          click: () => onZoneClick(zone.zone_id)
        }}
      >
      <Tooltip>
        <Box>
          <Typography variant="caption"
fontWeight="bold">
            {zone.zone_id}
          </Typography>
          <Typography variant="caption" display="block">
            Density: {zone.current_density.toFixed(2)}
            p/m2
          </Typography>
          <Typography variant="caption" display="block">
            Risk: {zone.congestion_risk}
          </Typography>
        </Box>
      </Tooltip>
    </Polygon>
  )})}

  {/* Incident Markers */}
  {incidents.map(incident => (
    <Marker
      key={incident.incident_id}

```

```

                position=[incident.location.latitude,
incident.location.longitude]}
                icon={getIncidentIcon(incident.severity,
incident.incident_type)}
            >
            <Popup>
                <Box>
                    <Typography variant="subtitle2"
fontWeight="bold">
                        {incident.incident_number}
                    </Typography>
                    <Typography variant="caption" display="block">
                        {incident.description}
                    </Typography>
                    <Button size="small" onClick={() =>
handleViewIncident(incident)}>
                        View Details
                    </Button>
                </Box>
            </Popup>
        </Marker>
    ))}

    {/* Staff Location Markers */}
    {staffLocations
        .filter(staff => staff.status === 'on_duty' ||
staff.status === 'responding')
        .map(staff => (
            <Marker
                key={staff.location_id}
                position=[staff.location.latitude,
staff.location.longitude]}
                icon={getStaffIcon(staff.status)}
            >
                <Tooltip>
                    Staff: {staff.status}
                </Tooltip>
            </Marker>
        ))}

```



```

        </Tooltip>
        </Marker>
      )))
    </MapContainer>
  </CardContent>
</Card>
);
};

// =====
// LIVE METRICS PANEL
// =====

interface LiveMetricsPanelProps {
  attendance: AttendanceMetrics;
  waitTimes: WaitTimeMetrics;
  transactions: TransactionMetrics;
}

const LiveMetricsPanel: React.FC<LiveMetricsPanelProps> = ({
  attendance,
  waitTimes,
  transactions
}) => {
  return (
    <Card>
      <CardHeader
        title="Live Metrics"
        sx={{ background: '#9AAF45', color: 'white' }}
      />
      <CardContent>
        <Stack spacing={2}>
          {/* Attendance */}
          <MetricCard
            title="Attendance"
            value={attendance.current_occupancy.toLocaleString()}

```

```

        subtitle={`\${attendance.capacity_percentage}%
capacity`}
        trend={attendance.entry_rate_per_minute > 0 ?
'increasing' : 'stable'}
        icon={<PeopleIcon />}
        color="#1B3A4B"
    />

    {/* Average Wait Times */}
    <Box>
        <Typography variant="subtitle2" gutterBottom>
            Average Wait Times
        </Typography>
        <Stack spacing={1}>
            <Box display="flex" justifyContent="space-between">
                <Typography variant="body2">Entry
Gates</Typography>
                <Chip
                    label={`\${waitTimes.avg_entry_wait}min`}
                    size="small"
                    color={waitTimes.avg_entry_wait > 10 ? 'error'
: 'success'}
                />
            </Box>
            <Box display="flex" justifyContent="space-between">
                <Typography
variant="body2">Concessions</Typography>
                <Chip
                    label={`\${waitTimes.avg_concession_wait}min`}
                    size="small"
                    color={waitTimes.avg_concession_wait > 5 ?
'warning' : 'success'}
                />
            </Box>
            <Box display="flex" justifyContent="space-between">

```

```

        <Typography
variant="body2">Restrooms</Typography>
        <Chip
            label={`$${waitTimes.avg_restroom_wait}min`}
            size="small"
            color={waitTimes.avg_restroom_wait > 3 ?
'warning' : 'success'}
        />
    </Box>
</Stack>
</Box>

{/* Transactions */}
<Box>
    <Typography variant="subtitle2" gutterBottom>
        Transactions (Last 15min)
    </Typography>
    <Grid container spacing={2}>
        <Grid item xs={6}>
            <Box textAlign="center">
                <Typography variant="h5" color="#9AAF45">
                    {transactions.count_15min}
                </Typography>
                <Typography
variant="caption">Count</Typography>
            </Box>
        </Grid>
        <Grid item xs={6}>
            <Box textAlign="center">
                <Typography variant="h5" color="#3A7B8A">

                    ${transactions.revenue_15min.toLocaleString()}
                </Typography>
                <Typography
variant="caption">Revenue</Typography>
            </Box>

```

```

        </Grid>
      </Grid>
    </Box>
  </Stack>
</CardContent>
</Card>
);
};

// =====
// INCIDENT DETAIL MODAL
// =====

interface IncidentDetailModalProps {
  incident: Incident;
  onClose: () => void;
  onUpdate: (incident: Incident) => void;
}

const IncidentDetailModal: React.FC<IncidentDetailModalProps> =
({
  incident,
  onClose,
  onUpdate
}) => {
  const [timeline, setTimeline] =
    useState<IncidentTimeline[]>([]);
  const [activeProtocol, setActiveProtocol] =
    useState<ResponseProtocol | null>(null);

  useEffect(() => {
    // Load incident timeline
    fetchTimeline(incident.incident_id);
    // Load active protocol if any
    fetchActiveProtocol(incident.incident_id);
  }, [incident.incident_id]);

```

```

return (
  <Dialog open fullWidth maxWidth="lg">
    <DialogTitle sx={{ background: '#1B3A4B', color: 'white'
  }}>
      <Box display="flex" justifyContent="space-between"
alignItems="center">
        <Box>
          <Typography
variant="h6">{incident.incident_number}</Typography>
          <Typography variant="caption">
            {incident.incident_type} • {incident.severity}
          </Typography>
        </Box>
        <IconButton onClick={onClose} sx={{ color: 'white' }}>
          <CloseIcon />
        </IconButton>
      </Box>
    </DialogTitle>

    <DialogContent dividers>
      <Grid container spacing={3}>
        {/* Left: Incident Details */}
        <Grid item xs={12} md={6}>
          <Stack spacing={2}>
            {/* Status & Actions */}
            <Card>
              <CardContent>
                <Typography variant="subtitle2" gutterBottom>
                  Status & Actions
                </Typography>
                <Box display="flex" gap={1} mb={2}>
                  <Chip
                    label={incident.status.toUpperCase()}
                    color={getStatusChipColor(incident.status)}
                  />

```

```

        <Chip
          label={incident.severity.toUpperCase()}
          color={getSeverityChipColor(incident.severity)}
        />
      </Box>

      <Stack spacing={1}>
        {incident.status === 'reported' && (
          <Button
            variant="contained"
            color="primary"
            onClick={() =>
handleAcknowledge(incident)}
          >
            Acknowledge & Respond
          </Button>
        )}
        {incident.status === 'acknowledged' && (
          <Button
            variant="contained"
            color="warning"
            onClick={() =>
handleActivateProtocol(incident)}
          >
            Activate Response Protocol
          </Button>
        )}
        {[ 'acknowledged',
'responding' ].includes(incident.status) && (
          <Button
            variant="contained"
            color="success"
            onClick={() => handleResolve(incident)}
          >
            Mark Resolved

```

```

        </Button>
    )}
    <Button
        variant="outlined"
        color="error"
        onClick={() => handleEscalate(incident)}
    >
        Escalate
    </Button>
</Stack>
</CardContent>
</Card>

{/* Location & Details */}
<Card>
    <CardContent>
        <Typography variant="subtitle2" gutterBottom>
            Location & Details
        </Typography>
        <Stack spacing={1}>
            <Box>
                <Typography variant="caption"
color="text.secondary">
                    Location
                </Typography>
                <Typography variant="body2">
                    {incident.location.zone_id} - Section
                    {incident.location.section}
                    {incident.location.row && ``, Row
                    ${incident.location.row}`}
                </Typography>
            </Box>
            <Box>
                <Typography variant="caption"
color="text.secondary">
                    Description

```

```

        </Typography>
        <Typography variant="body2">
            {incident.description}
        </Typography>
    </Box>
    <Box>
        <Typography variant="caption"
color="text.secondary">
            Reported By
        </Typography>
        <Typography variant="body2">
            {incident.reporter.reporter_type}
        </Typography>
    </Box>
</Stack>
</CardContent>
</Card>

{ /* Response Team */ }
<Card>
    <CardContent>
        <Typography variant="subtitle2" gutterBottom>
            Response Team
        </Typography>
        <Stack spacing={1}>
            <Box display="flex"
justifyContent="space-between">
                <Typography
variant="body2">Team:</Typography>
                <Typography variant="body2"
fontWeight="bold">
                    {incident.response_team || 'Not
assigned' }
                </Typography>
            </Box>

```



```

                <Box display="flex"
justifyContent="space-between">
                    <Typography variant="body2">Staff
Assigned:</Typography>
                    <Typography variant="body2"
fontWeight="bold">
                        {incident.assigned_to.length}
                    </Typography>
                </Box>
            </Stack>
        </CardContent>
    </Card>
</Stack>
</Grid>

{/* Right: Timeline & Protocol */}
<Grid item xs={12} md={6}>
    <Stack spacing={2}>
        {/* Active Protocol */}
        {activeProtocol && (
            <Card>
                <CardHeader
                    title="Active Response Protocol"
                    sx={{ background: '#D0021B', color: 'white'
}}}
                />
                <CardContent>
                    <Typography variant="subtitle2" gutterBottom>
                        {activeProtocol.protocol_name}
                    </Typography>
                    <LinearProgress
                        variant="determinate"

value={calculateProtocolProgress(activeProtocol)}
                        sx={{ mb: 2 }}
                    />

```

```

        <List dense>
          {activeProtocol.protocol_steps.map((step,
index) => (
            <ListItem key={index}>
              <ListItemIcon>
                {step.status === 'completed' ? (
                  <CheckCircleIcon color="success" />
                ) : step.status === 'in_progress' ? (
                  <CircularProgress size={20} />
                ) : (
                  <RadioButtonUncheckedIcon />
                )}
              </ListItemIcon>
              <ListItemText
                primary={step.step_name}
                secondary={step.assigned_to}
              />
            </ListItem>
          )}}
        </List>
      </CardContent>
    </Card>
  )}

  {/* Timeline */}
  <Card>
    <CardHeader title="Incident Timeline" />
    <CardContent sx={{ maxHeight: '400px', overflow:
'auto' }}>
      <Timeline>
        {timeline.map((entry, index) => (
          <TimelineItem key={entry.timeline_id}>
            <TimelineSeparator>
              <TimelineDot
color={getTimelineColor(entry.event_type)} />

```

```

                                {index < timeline.length - 1 &&
<TimelineConnector />}}
                                </TimelineSeparator>
                                <TimelineContent>
                                    <Typography variant="subtitle2">
                                        {entry.event_type.replace('_', '
').toUpperCase()}}
                                    </Typography>
                                    <Typography variant="caption"
color="text.secondary">
                                        {formatTimestamp(entry.timestamp)} •
{entry.actor.actor_name}
                                    </Typography>
                                    <Typography
variant="body2">{entry.details}</Typography>
                                        {entry.evidence_captured && (
                                            <Box mt={1}>
                                                <Chip label="Evidence Captured"
size="small" icon={<CameraIcon />} />
                                            </Box>
                                        )}
                                    </TimelineContent>
                                </TimelineItem>
                            )}}
                        </Timeline>
                    </CardContent>
                </Card>
            </Stack>
        </Grid>
    </Grid>
</DialogContent>

    <DialogActions>
        <Button onClick={onClose}>Close</Button>
        <Button variant="contained" onClick={() =>
handleAddTimelineEntry(incident)}>

```

```
        Add Update
      </Button>
    </DialogActions>
  </Dialog>
);
};
```

4. TEMPORAL SEQUENCE

typescript

```
TypeScript
// =====
// RUN_ZONE_TEMPORAL_SEQUENCE
// =====

const RUN_TEMPORAL_SEQUENCE = {
  "T-0_GATES_OPEN": {
    phase: "Ingress",
    key_activities: [
      "Guest entry processing",
      "Security screening",
      "Parking management",
      "Initial crowd flow monitoring"
    ],
    monitoring_activated: [
      "Entry gate throughput",
      "Security checkpoint wait times",
      "Parking occupancy",
      "Initial crowd density in common areas"
    ],
    typical_duration: "90-120 minutes",
    critical_metrics: {
      entry_rate_target: "500-1000 guests/minute",
      avg_wait_time_target: "< 5 minutes",
```

```

    security_screening_rate: "< 30 seconds per guest"
  },
  automated_responses: [
    {
      trigger: "Entry wait time > 10 minutes",
      action: "Open additional entry lanes",
      notification: ["Operations Manager", "Guest Services"]
    },
    {
      trigger: "Security checkpoint backup > 50 people",
      action: "Deploy additional screening staff",
      notification: ["Security Chief"]
    }
  ]
},

"T+30_PRE_EVENT": {
  phase: "Pre-Event / Fan Experience",
  key_activities: [
    "Concession operations at peak",
    "Merchandise sales",
    "Pre-game entertainment",
    "Final seat finding"
  ],
  monitoring_activated: [
    "Concession wait times",
    "Restroom queue lengths",
    "Merchandise transaction rates",
    "Seating area congestion"
  ],
  typical_duration: "30-60 minutes",
  critical_metrics: {
    concession_wait_target: "< 5 minutes",
    transaction_success_rate: "> 98%",
    guest_satisfaction_sampling: "Real-time surveys"
  },

```

```
    automated_responses: [  
      {  
        trigger: "Concession wait time > 8 minutes",  
        action: "Alert concessions manager for staffing  
adjustment",  
        notification: ["Concessions Manager", "Operations"]  
      },  
      {  
        trigger: "Restroom queue > 15 people",  
        action: "Increase cleaning frequency, notify guests of  
alternate locations",  
        notification: ["Facilities", "Guest Services"]  
      }  
    ]  
  },
```

```
"T+KICKOFF_LIVE_EVENT": {  
  phase: "Live Event",  
  key_activities: [  
    "Real-time event monitoring",  
    "Incident response",  
    "Emergency protocol readiness",  
    "Guest service requests"  
  ],  
  monitoring_activated: [  
    "Crowd behavior analysis",  
    "Medical incidents",  
    "Weather conditions",  
    "System performance under load",  
    "Security threats"  
  ],  
  typical_duration: "3-4 hours",  
  critical_metrics: {  
    incident_response_time: "< 2 minutes",  
    system_availability: "99.9%+",  
    medical_response_time: "< 90 seconds",
```

```
    crowd_density_thresholds: "0.8 people/sq meter max"
  },
  emergency_scenarios: [
    {
      scenario: "Medical Emergency",
      response_time_target: "90 seconds",
      protocol: "MEDICAL_EMERGENCY",
      resources: ["Medical team", "AED", "Ambulance
coordination"]
    },
    {
      scenario: "Weather Alert (Lightning)",
      response_time_target: "30 seconds",
      protocol: "LIGHTNING_EVACUATION",
      resources: ["All staff", "PA system", "Mobile alerts"]
    },
    {
      scenario: "Security Threat",
      response_time_target: "60 seconds",
      protocol: "SECURITY_LOCKDOWN",
      resources: ["Security team", "Law enforcement",
"Communication systems"]
    }
  ],
  automated_responses: [
    {
      trigger: "Crowd density > 0.8 people/sq meter",
      action: "Alert operations, deploy crowd management,
adjust flow",
      notification: ["Operations Manager", "Security Chief"]
    },
    {
      trigger: "System degradation detected",
      action: "Auto-failover to backup systems, alert IT",
      notification: ["IT Manager", "Operations"]
    }
  ]
}
```

```

    ]
  },

  "T+HALFTIME": {
    phase: "Halftime",
    key_activities: [
      "Peak concession demand",
      "Restroom surge",
      "Maintenance quick checks",
      "Halftime show coordination"
    ],
    monitoring_activated: [
      "Concession transaction spike",
      "Restroom occupancy surge",
      "Crowd flow to/from seating",
      "Equipment performance check"
    ],
    typical_duration: "12-20 minutes",
    critical_metrics: {
      concession_throughput: "2x normal rate",
      restroom_turnaround: "< 2 minutes per person",
      return_to_seats: "90% by 2nd half kickoff"
    },
    automated_responses: [
      {
        trigger: "Halftime start",
        action: "Auto-alert all concession and facilities staff for surge",
        notification: ["Concessions", "Facilities", "Guest Services"]
      },
      {
        trigger: "Extended wait times > 10 minutes",
        action: "Deploy roving staff, open backup stations",
        notification: ["Operations Manager"]
      }
    ]
  }
}

```



```

    ]
  },

  "T+EVENT_END_EGRESS": {
    phase: "Egress / Exit",
    key_activities: [
      "Controlled crowd exit",
      "Parking egress management",
      "Final safety sweeps",
      "Lost & found processing"
    ],
    monitoring_activated: [
      "Exit flow rates",
      "Parking lot congestion",
      "Stragglers in venue",
      "Traffic coordination"
    ],
    typical_duration: "45-90 minutes",
    critical_metrics: {
      venue_clear_time: "< 45 minutes for 90% of guests",
      parking_exit_rate: "600-800 vehicles/hour",
      safety_incidents: "Zero tolerance"
    },
    automated_responses: [
      {
        trigger: "Exit bottleneck detected",
        action: "Open additional exit gates, adjust traffic
signals",
        notification: ["Operations", "Traffic Management"]
      },
      {
        trigger: "Venue 80% cleared",
        action: "Initiate final safety sweep, prepare for
closure",
        notification: ["Security", "Operations"]
      }
    ]
  }
}

```

```

    ]
  },

  "T+POST_EVENT_CLOSURE": {
    phase: "Post-Event Wrap-Up",
    key_activities: [
      "Final venue sweep",
      "Equipment shutdown procedures",
      "Incident report compilation",
      "Evidence ledger finalization"
    ],
    monitoring_activated: [
      "All systems shutdown status",
      "Security zone clearance",
      "Lost & found items logged"
    ],
    typical_duration: "60-120 minutes",
    transition_to: "REVIEW zone",
    deliverables: [
      "Event summary report",
      "Incident logs",
      "Evidence ledger sealed",
      "Initial after-action notes"
    ]
  }
};

```

5. STATE MACHINE

typescript

```

TypeScript
// =====
// RUN ZONE STATE MACHINE
// =====

```

```

type RunState =
  | 'INGRESS'           // T-0 to T+30 (gates open, entry)
  | 'PRE_EVENT'         // T+30 to kickoff (pre-game)
  | 'LIVE_EVENT'        // Kickoff to end (active event)
  | 'HALFTIME'          // Halftime period
  | 'POST_EVENT'        // Event end to venue clear
  | 'EGRESS'            // Controlled exit
  | 'CLOSURE'           // Final wrap-up
  | 'RUN_COMPLETE';     // Terminal state

interface RunStateMachine {
  current_state: RunState;
  venue_id: string;
  event_id: string;
  gates_opened_at: timestamp;
  event_started_at: timestamp;
  active_incidents: number;
  critical_alerts: number;

  can_transition_to(target_state: RunState):
  TransitionValidation;
  transition_to(target_state: RunState): TransitionResult;
}

const RUN_STATE_TRANSITIONS: Record<RunState, {
  next_states: RunState[];
  entry_conditions: Condition[];
  on_entry: Action[];
  on_exit: Action[];
}> = {

  INGRESS: {
    next_states: ['PRE_EVENT'],
    entry_conditions: [
      {

```

```

        condition: 'Gates opened',
        validation: (context) => context.gates_opened_at !== null
    },
    {
        condition: 'All systems operational',
        validation: (context) => context.system_health_score >=
98    }
    ],
    on_entry: [
        'Activate live monitoring',
        'Enable entry tracking',
        'Start crowd flow analysis',
        'Arm emergency protocols'
    ],
    on_exit: [
        'Log ingress completion metrics',
        'Verify 80% capacity reached'
    ]
},

PRE_EVENT: {
    next_states: ['LIVE_EVENT'],
    entry_conditions: [
        {
            condition: 'Sufficient attendance',
            validation: (context) => context.current_occupancy >=
context.expected_attendance * 0.8
        }
    ],
    on_entry: [
        'Peak concession monitoring',
        'Guest experience tracking',
        'Pre-event entertainment coordination'
    ],
    on_exit: [

```

```

        'Prepare for event start',
        'Final safety checks'
    ]
},

LIVE_EVENT: {
    next_states: ['HALFTIME', 'POST_EVENT'],
    entry_conditions: [
        {
            condition: 'Event started',
            validation: (context) => context.event_started_at !==
null
        }
    ],
    on_entry: [
        'Maximum alertness mode',
        'Real-time incident tracking',
        'Continuous crowd behavior analysis',
        'Emergency response teams on standby'
    ],
    on_exit: [
        'Log event phase metrics'
    ]
},

HALFTIME: {
    next_states: ['LIVE_EVENT'],
    entry_conditions: [
        {
            condition: 'Halftime triggered',
            validation: (context) => context.event_phase ===
'halftime'
        }
    ],
    on_entry: [
        'Alert all staff for surge demand',

```

```

        'Heightened concession monitoring',
        'Restroom capacity tracking'
    ],
    on_exit: [
        'Verify return to seats',
        'Resume event monitoring'
    ]
},

POST_EVENT: {
    next_states: ['EGRESS'],
    entry_conditions: [
        {
            condition: 'Event concluded',
            validation: (context) => context.event_ended_at !== null
        }
    ],
    on_entry: [
        'Initiate egress protocols',
        'Activate traffic management',
        'Prepare exit monitoring'
    ],
    on_exit: [
        'Verify egress started',
        'Monitor exit flow rates'
    ]
},

EGRESS: {
    next_states: ['CLOSURE'],
    entry_conditions: [
        {
            condition: 'Egress started',
            validation: (context) => context.egress_started === true
        }
    ],

```

```

    on_entry: [
        'Full exit monitoring',
        'Parking egress coordination',
        'Lost guest protocols active'
    ],
    on_exit: [
        'Verify 90% venue cleared',
        'Prepare for final sweep'
    ]
},

CLOSURE: {
    next_states: ['RUN_COMPLETE'],
    entry_conditions: [
        {
            condition: 'Venue substantially cleared',
            validation: (context) => context.current_occupancy <
context.expected_attendance * 0.1
        }
    ],
    on_entry: [
        'Final safety sweep',
        'Equipment shutdown',
        'Compile incident reports',
        'Seal evidence ledger'
    ],
    on_exit: [
        'Generate event summary',
        'Transition to REVIEW zone'
    ]
},

RUN_COMPLETE: {
    next_states: [], // Terminal state
    entry_conditions: [
        {

```

```

        condition: 'All closure tasks complete',
        validation: (context) => context.closure_complete ===
true
    }
],
on_entry: [
    'Archive RUN zone data',
    'Prepare REVIEW zone activation'
],
on_exit: []
}
};
```

```

---

This completes **ZONE 3: RUN** specifications at the same technical depth as ZONE 1 and ZONE 2.

Given the length and detail we've covered, here's the **continuation prompt for ZONE 4: REVIEW**:

---

```

CONTINUATION PROMPT FOR NEW THREAD
```

```

CONTEXT: We've completed detailed technical specifications for eVenu360 MVP ZONES 1-3:

- ZONE 1: PREPARE (Blueprint & Compliance) 
- ZONE 2: READY (Pre-Event Validation) 
- ZONE 3: RUN (Live Operations & Response) 

NOW I NEED: Complete ZONE 4: REVIEW with the same technical depth.

ZONE 4: REVIEW (After-Action & Continuous Improvement)

Temporal Frame: T+0 (Post-Event) to T+30 days

Please provide:

1. DATABASE SCHEMA - Tables for after-action reports, performance analytics, lessons learned, improvement recommendations, benchmarking data
2. API ENDPOINTS - Auto-generated reports, evidence ledger queries, performance analysis, cross-event comparisons, improvement workflow
3. UI COMPONENTS - After-action dashboard, analytics interface, evidence viewer, report builder, improvement tracking
4. TEMPORAL SEQUENCE - Post-event timeline: immediate (T+0 to T+24hr), analysis (T+1 to T+7 days), integration (T+7 to T+30 days)
5. STATE MACHINE - REVIEW zone states from data collection → analysis → recommendations → implementation → closure (loops back to PREPARE for next event)

MAINTAIN:

- Same database naming conventions
- Consistent API patterns
- TypeScript interfaces
- Sentrais brand colors (navy #1B3A4B, teal #3A7B8A, green #9AAF45)
- Integration with Evidence Ledger and SIPE Engine
- Connection points back to PREPARE zone (continuous improvement loop)

START WITH: Complete database schema for ZONE 4: REVIEW